

Nama :Elsa Marthalinda

NIM :254107020204

Kelas :TI – 1G

LAPORAN JOBSHEET 12

12.2 Percobaan 1: Operasi Penambahan pada Double Linked List

Kode Program MahasiswaDLL7.java;

```
package Jobsheet12;
public class MahasiswaDLL7 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public MahasiswaDLL7 (String nim, String nama, String kelas, double
ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }
    public void tampil() {
        System.out.println(
            "NIM      : " + nim +
            "\nNama    : " + nama +
            "\nKelas  : " + kelas +
            "\nIPK     : " + ipk
        );
        System.out.println("-----");
    }
}
```

Kode Program NodeDLL7.java;

```
package Jobsheet12;
public class NodeDLL7 {
    MahasiswaDLL7 data;
    NodeDLL7 prev;
    NodeDLL7 next;

    public NodeDLL7(MahasiswaDLL7 data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}
```

Kode Program DoubleLinkedList7.java;

```
package Jobsheet12;
public class DoubleLinkedList7 {
    NodeDLL7 head;
    NodeDLL7 tail;

    public DoubleLinkedList7() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(MahasiswaDLL7 data) {
        NodeDLL7 newNode = new NodeDLL7(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    public void addLast(MahasiswaDLL7 data) {
        NodeDLL7 newNode = new NodeDLL7(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    public void insertAfter (String keyNim, MahasiswaDLL7 data) {
        NodeDLL7 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }
        if (current == null) {
            System.out.println("Data dengan NIM " + keyNim + " tidak
ditemukan.");
            return;
        }
        NodeDLL7 newNode = new NodeDLL7(data);

        if(current == tail) {
            newNode.prev = current;
            current.next = newNode;
            tail = newNode;
        } else {
            newNode.prev = current;
            newNode.next = current.next;
        }
    }
}
```

```

        current.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " +
keyNim);
}

    public void print() {
        if (isEmpty()) {
            System.out.println("Linked List maasih kosong.");
            return;
        }
        NodeDLL7 current = head;
        while (current != null) {
            current.data.tampil();
            current = current.next;
        }
    }
}

```

Kode Program DoubleLinkedListMain7.java;

```

package Jobsheet12;
import java.util.Scanner;
public class DoubleLinkedListMain7 {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        DoubleLinkedList7 list = new DoubleLinkedList7();
        int pilihan;

        do {
            System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");
            System.out.println("3. Sisipkan data di tengah (setelah
NIM)");
            System.out.println("4. Hapus data di awal");
            System.out.println("5. Hapus data di akhir");
            System.out.println("6. Tampilkan data");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = scan.nextInt();
            scan.nextLine();

            switch (pilihan) {
                case 1:
                    MahasiswaDLL7 mhsAwal = inputMahasiswaDLL7(scan);
                    list.addFirst(mhsAwal);
                    break;
                case 2:
                    MahasiswaDLL7 mhsAkhir = inputMahasiswaDLL7(scan);
                    list.addLast(mhsAkhir);
                    break;
                case 3:
                    System.out.print("Masukkan NIM yang dicari: ");
                    String keyNim = scan.nextLine();

```

```

        System.out.println("Masukkan data baru: ");
        MahasiswaDLL7 dataBaru = inputMahasiswaDLL7(scan);
        list.insertAfter(keyNim, dataBaru);
        break;
// case 4:
//     list.removeFirst();
//     break;
// case 5:
//     list.removeLast();
//     break;
case 6:
    list.print();
    break;
case 0:
    System.out.println("Program selesai.");
    break;
default:
    System.out.println("Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();
}
public static MahasiswaDLL7 inputMahasiswaDLL7(Scanner scan) {
    System.out.print("Masukkan NIM : ");
    String nim = scan.nextLine();
    System.out.print("Masukkan Nama : ");
    String nama = scan.nextLine();
    System.out.print("Masukkan Kelas: ");
    String kelas = scan.nextLine();
    System.out.print("Masukkan IPK : ");
    double ipk = scan.nextDouble();
    scan.nextLine();

    return new MahasiswaDLL7(nim, nama, kelas, ipk);
}
}

```

Hasil Output:

===== MENU DOUBLE LINKED LIST =====

```

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 2
Masukkan NIM : 123005
Masukkan Nama : Harry
Masukkan Kelas: 1A
Masukkan IPK : 3,76

```

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar

Pilih menu: 3

Masukkan NIM yang dicari: 123005

Masukkan data baru:

Masukkan NIM : 123010

Masukkan Nama : Potter

Masukkan Kelas: 1B

Masukkan IPK : 3,55

Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar

Pilih menu: 6

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

12.2.3 Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!
 - Single Linked List hanya memiliki satu pointer yaitu next yang menunjuk ke node berikutnya sehingga traversal hanya dapat dilakukan dari depan ke belakang
 - Double Linked List memiliki dua pointer yaitu next dan prev, sehingga traversal dapat dilakukan dua arah, yaitu dari head ke tail maupun dari tail ke head.
2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!
 - Atribut next berfungsi untuk menyimpan alamat node berikutnya sehingga traversal dapat bergerak maju ke depan.
 - Atribut prev berfungsi untuk menyimpan alamat node sebelumnya sehingga traversal dapat bergerak mundur ke belakang.
3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!
 - Konstruktor pada class DoubleLinkedList7 berfungsi untuk menginisialisasi kondisi awal linked list dengan mengatur head dan tail bernilai null. Hal ini menandakan bahwa linked list masih kosong dan belum memiliki node sama sekali.
4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?
 - Karena saat linked list masih kosong dan node pertama ditambahkan, maka node tersebut menjadi satu-satunya node dalam linked list. Oleh sebab itu, head dan tail harus menunjuk ke node yang sama karena node pertama tersebut sekaligus menjadi node awal dan node akhir.
5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!

===== MENU DOUBLE LINKED LIST =====

```

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu: 6
Linked List masih kosong.

```

6. Modifikasi kode program dengan menambahkan method **printReverse()** untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

➤ **Kode Program DoubleLinkedList7.java;**

```

public void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong.");
        return;
    }
    NodeDLL7 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

```

➤ **Kode Program DoubleLinkedListMain7.java;**

```

System.out.println("7. Tampilkan data secara terbalik");

case 7:
    list.printReverse();
    break;

```

➤ **Hasil Output:**

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik

```

0. Keluar
Pilih menu: 7
NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55

NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76

12.3 Percobaan 2: Operasi Penghapusan pada Double Linked List

Kode Program DoubleLinkedList7.java;

```
package Jobsheet12;
public class DoubleLinkedList7 {
    NodeDLL7 head;
    NodeDLL7 tail;

    public DoubleLinkedList7() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(MahasiswaDLL7 data) {
        NodeDLL7 newNode = new NodeDLL7(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    public void addLast(MahasiswaDLL7 data) {
        NodeDLL7 newNode = new NodeDLL7(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }
}
```



```

public void insertAfter (String keyNim, MahasiswaDLL7 data) {
    NodeDLL7 current = head;
    while (current != null && !current.data.nim.equals(keyNim)) {
        current = current.next;
    }
    if (current == null) {
        System.out.println("Data dengan NIM " + keyNim + " tidak
ditemukan.");
        return;
    }
    NodeDLL7 newNode = new NodeDLL7(data);

    if(current == tail) {
        newNode.prev = current;
        current.next = newNode;
        tail = newNode;
    } else {
        newNode.prev = current;
        newNode.next = current.next;
        current.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " +
keyNim);
}

public void print() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }
    NodeDLL7 current = head;
    while (current != null) {
        current.data.tampil();
        current = current.next;
    }
}

public void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong.");
        return;
    }
    NodeDLL7 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
    if (head == tail) {

```

```

        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
}
public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}
}

```

Kode program DoubleLinkedListMain7.java;

```

package Jobsheet12;
import java.util.Scanner;
public class DoubleLinkedListMain7 {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        DoubleLinkedList7 list = new DoubleLinkedList7();
        int pilihan;

        do {
            System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");
            System.out.println("3. Sisipkan data di tengah (setelah
NIM)");
            System.out.println("4. Hapus data di awal");
            System.out.println("5. Hapus data di akhir");
            System.out.println("6. Tampilkan data");
            System.out.println("7. Tampilkan data secara terbalik");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = scan.nextInt();
            scan.nextLine();

            switch (pilihan) {
                case 1:
                    MahasiswaDLL7 mhsAwal = inputMahasiswaDLL7(scan);
                    list.addFirst(mhsAwal);
                    break;
                case 2:
                    MahasiswaDLL7 mhsAkhir = inputMahasiswaDLL7(scan);
                    list.addLast(mhsAkhir);
                    break;
                case 3:
                    System.out.print("Masukkan NIM yang dicari: ");
                    String keyNim = scan.nextLine();

```

```

        System.out.println("Masukkan data baru: ");
        MahasiswaDLL7 dataBaru = inputMahasiswaDLL7(scan);
        list.insertAfter(keyNim, dataBaru);
        break;
    case 4:
        list.removeFirst();
        break;
    case 5:
        list.removeLast();
        break;
    case 6:
        list.print();
        break;
    case 7:
        list.printReverse();
        break;
    case 0:
        System.out.println("Program selesai.");
        break;
    default:
        System.out.println("Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();
}

public static MahasiswaDLL7 inputMahasiswaDLL7(Scanner scan) {
    System.out.print("Masukkan NIM : ");
    String nim = scan.nextLine();
    System.out.print("Masukkan Nama : ");
    String nama = scan.nextLine();
    System.out.print("Masukkan Kelas: ");
    String kelas = scan.nextLine();
    System.out.print("Masukkan IPK : ");
    double ipk = scan.nextDouble();
    scan.nextLine();

    return new MahasiswaDLL7(nim, nama, kelas, ipk);
}
}

```

Hasil Output:

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu: 2

Masukkan NIM : 123005

Masukkan Nama : Harry

Masukkan Kelas: 1A

Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu: 3

Masukkan NIM yang dicari: 123005

Masukkan data baru:

Masukkan NIM : 123010

Masukkan Nama : Potter

Masukkan Kelas: 1B

Masukkan IPK : 3,55

Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu: 6

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
 2. Tambah data di akhir
 3. Sisipkan data di tengah (setelah NIM)
 4. Hapus data di awal
 5. Hapus data di akhir
 6. Tampilkan data
 7. Tampilkan data secara terbalik
 0. Keluar
- Pilih menu: 5

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
 2. Tambah data di akhir
 3. Sisipkan data di tengah (setelah NIM)
 4. Hapus data di awal
 5. Hapus data di akhir
 6. Tampilkan data
 7. Tampilkan data secara terbalik
 0. Keluar
- Pilih menu: 6
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76
-

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
 2. Tambah data di akhir
 3. Sisipkan data di tengah (setelah NIM)
 4. Hapus data di awal
 5. Hapus data di akhir
 6. Tampilkan data
 7. Tampilkan data secara terbalik
 0. Keluar
- Pilih menu: 0
Program selesai.

12.3.3 Pertanyaan Percobaan

1. Perhatikan potongan kode berikut pada method removeFirst():

```
head = head.next;  
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

- `head = head.next;` Statement ini berfungsi untuk memindahkan penunjuk head ke node setelahnya (node kedua). Dengan demikian, node kedua kini resmi menjadi node pertama yang baru di dalam list.
 - `head.prev = null;` Setelah penunjuk head berpindah ke node baru, statement ini memutuskan hubungan ke node lama dengan mengatur atribut prev pada node pertama yang baru menjadi null. Hal ini memastikan tidak ada lagi referensi yang menunjuk ke node yang telah dihapus sehingga node tersebut dapat dibersihkan oleh garbage collector.
2. Modifikasi method `removeFirst()` dan `removeLast()` agar program menampilkan data yang berhasil dihapus.

- **Kode Program** `DoubleLinkedList7.java`;

```
public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    } else {
        System.out.println("Data berhasil dihapus.");
        head.data.tampil();
    }
    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
}

public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    } else {
        System.out.println("Data berhasil dihapus.");
        tail.data.tampil();
    }
    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}
```

Hasil Output:

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu: 2

Masukkan NIM : 123005

Masukkan Nama : Harry

Masukkan Kelas: 1A

Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu: 3

Masukkan NIM yang dicari: 123005

Masukkan data baru:

Masukkan NIM : 123010

Masukkan Nama : Potter

Masukkan Kelas: 1B

Masukkan IPK : 3,55

Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik

0. Keluar

Pilih menu: 6

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal

2. Tambah data di akhir

3. Sisipkan data di tengah (setelah NIM)

4. Hapus data di awal

5. Hapus data di akhir

6. Tampilkan data

7. Tampilkan data secara terbalik

0. Keluar

Pilih menu: 5

Data berhasil dihapus.

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal

2. Tambah data di akhir

3. Sisipkan data di tengah (setelah NIM)

4. Hapus data di awal

5. Hapus data di akhir

6. Tampilkan data

7. Tampilkan data secara terbalik

0. Keluar

Pilih menu: 6

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu: 0

Program selesai.